LANGCHAIN — DOCUMENT CLASS

Complete Teaching Notes with Theory, Examples, Python Programs and MCQs

1. Introduction to Document Loaders

Document Loaders help us read supported external sources and represent their content as LangChain Document objects.

2. What Is a LangChain Document?

A LangChain Document is a class used to represent a piece of text and its associated metadata.

A Document mainly contains two important attributes:

1. **page_content** — Stores the actual text.
2. **metadata** — Stores additional information about the text.

3. First Important Attribute — page_content

page_content stores the actual text represented by a LangChain Document.

Example

Suppose a school notice says:

"All students must carry their ID cards."

Question: Where should we maintain this text?

Answer: Inside page_content.

Important Observations

1. page_content contains text.
2. It can contain a single sentence or multiple paragraphs.
3. The word page does not mean exactly one PDF page.
4. page_content does not automatically generate an AI answer.

Important Conclusion

page_content = WHAT IS WRITTEN.

4. Second Important Attribute — metadata

Metadata is additional information describing the content.

4.1 Five Important Technical Observations

Observation 1 — Metadata Is a Dictionary

Metadata is represented as a Python dictionary.

Example:

```
metadata = {
   "source": "student_notice.txt",
```

"department": "Administration",

"year": 2026

}

Observation 2 — Metadata Is Optional

If not supplied, its default value is an empty dictionary {}.

Observation 3 — Metadata Fields Are Not Fixed

Different Document objects may contain different keys.

Observation 4 — Custom Metadata Is Allowed

We can use our own custom metadata fields depending on the application.

Observation 5 — Metadata Does Not Read Files

If we manually enter a filename inside metadata, LangChain does not automatically read that file.

Important Conclusion

metadata = INFORMATION ABOUT THE CONTENT.

4.2 Classroom Questions

Question 1: Friends, where should I store the actual employee leave policy?

Expected Answer: page_content

Question 2: Where should I store the document's filename?

Expected Answer: metadata

Question 3: Where should I store the department name?

Expected Answer: metadata

Excellent!

5. Does One Document Always Mean One Physical File?

Friends, let us remove one important misunderstanding.

Suppose I have one PDF containing 100 pages.

Question: Will LangChain always create exactly one Document object?

Answer: No.

It depends on how the PDF is loaded and represented.

Three Possible Situations

Situation 1: One text file may be represented as one Document object.

Situation 2: One PDF may produce one Document object for every page.

Situation 3: One part of a larger file may also be represented as a separate Document object.

Important Conclusion

A LangChain Document represents a unit of information, not necessarily one complete physical file.

6. Important Document Class Syntax

6.1 Importing the Document Class

```
from langchain_core.documents import Document
```

Document is the class we will use.

6.2 Creating a Document with Text

```
document = Document(
    page_content="Python is a programming language."
)
```

6.3 Creating a Document with Metadata

```
document = Document(
    page_content="Python is a programming language.",
    metadata={
        "source": "python_notes.txt",
        "department": "Training"
```

```
    }
)
```

6.4 Accessing Document Attributes

Requirement	Syntax
Actual text	document.page_content
Complete metadata	document.metadata
Source information	document.metadata["source"]
Optional author field	document.metadata.get("author", "Not available")

Important Note

Using .get() with a default value helps us handle missing fields without a KeyError.

6.5 Five Important Observations Before Programming

1. Document is a LangChain class.
2. page_content stores the text.
3. metadata stores additional descriptive information.
4. Multiple Document objects can be maintained together.

5. Creating a Document does not automatically read a file or call an LLM.

7. Python Practical Programs

7.1 Program 1 — Creating the First LangChain Document

Objective: Create a Document object and access its content and metadata.

Complete Python Program

```
from langchain_core.documents import Document



# ========================================

# CREATE DOCUMENT OBJECT

# ========================================


document = Document(

    page_content="Python is a popular programming language."

)



# ========================================
```

```
# ACCESS PAGE CONTENT
# ========================================

print("PAGE CONTENT")
print("------------")
print(document.page_content)



# ========================================
# ACCESS METADATA
# ========================================

print("\nMETADATA")
print("--------")
print(document.metadata)



# ========================================
# CHECK OBJECT TYPE
# ========================================
```

```
print("\nOBJECT TYPE")

print("-----------")

print(type(document).__name__)
```

Important Note

A LangChain Document can be created directly using text, without reading an external file.

7.2 Program 2 — Document with Content and Metadata

Objective: Maintain actual text with source, department and year information.

Complete Python Program

```
from langchain_core.documents import Document



# ========================================

# CREATE DOCUMENT WITH METADATA

# ========================================


document = Document(

    page_content="The office opens at 9 AM.",
```

```
    metadata={
        "source": "office_rules.txt",
        "department": "Administration",
        "year": 2026
    }
)



# ========================================
# ACCESS PAGE CONTENT
# ========================================

print("PAGE CONTENT")
print("------------")
print(document.page_content)



# ========================================
# ACCESS COMPLETE METADATA
# ========================================
```

```
print("\nCOMPLETE METADATA")
print("----------------")
print(document.metadata)


# ========================================
# ACCESS INDIVIDUAL METADATA VALUES
# ========================================

print("\nSOURCE")
print("------")
print(document.metadata["source"])

print("\nDEPARTMENT")
print("----------")
print(document.metadata["department"])

print("\nYEAR")
print("----")
```

```
print(document.metadata["year"])
```

Important Note

A LangChain Document keeps actual content and descriptive information together inside one object.

7.3 Program 3 — Handling Missing Metadata

Objective: Safely access metadata fields that may not exist.

Complete Python Program

```
from langchain_core.documents import Document


# ========================================
# CREATE DOCUMENT
# ========================================

document = Document(
    page_content="Students must carry their ID cards.",
    metadata={
        "source": "college_notice.txt",
```

```
        "department": "Student Services"
    }
)


# ========================================
# ACCESS EXISTING METADATA
# ========================================

print("SOURCE")
print("------")
print(document.metadata["source"])

print("\nDEPARTMENT")
print("----------")
print(document.metadata.get("department"))


# ========================================
# ACCESS MISSING METADATA SAFELY
```

```
# ========================================


print("\nAUTHOR")
print("------")


print(
    document.metadata.get(
        "author",
        "Not available"
    )
)
```

Important Note

Metadata fields may differ between Document objects. We can use .get() to handle missing fields safely.

7.4 Program 4 — Working with Multiple Document Objects

Objective: Create several Documents, maintain them in one list and display their information.

Complete Python Program

```
from langchain_core.documents import Document
```

```
# ========================================
# CREATE FIRST DOCUMENT
# ========================================

document1 = Document(
    page_content="The library opens at 8 AM.",
    metadata={
        "source": "library_rules.txt"
    }
)


# ========================================
# CREATE SECOND DOCUMENT
# ========================================

document2 = Document(
    page_content="Students must wear their ID cards.",
    metadata={
```

```
        "source": "student_rules.txt"
    }
)



# ========================================
# CREATE THIRD DOCUMENT
# ========================================

document3 = Document(
    page_content="Fees must be paid before the deadline.",
    metadata={
        "source": "fee_rules.txt"
    }
)



# ========================================
# STORE DOCUMENTS IN A LIST
# ========================================
```

```
documents = [
    document1,
    document2,
    document3
]


# ========================================
# COUNT DOCUMENTS
# ========================================

print("TOTAL DOCUMENTS:", len(documents))


# ========================================
# DISPLAY DOCUMENT INFORMATION
# ========================================

for number, document in enumerate(
```

```
    documents,
    start=1
):

    print("\nDOCUMENT", number)

    print(
        "CONTENT:",
        document.page_content
    )

    print(
        "SOURCE:",
        document.metadata["source"]
    )
```

Important Note

One Document represents one unit of information. A list can maintain many Document objects together.

7.5 Program 5 — Master Program

Objective: Create multiple student information documents and display their content and metadata.

Complete Python Program

```
from langchain_core.documents import Document



# ========================================
# CREATE STUDENT INFORMATION DOCUMENTS
# ========================================

documents = [

    Document(
        page_content="Admissions start on 1 November 2026.",
        metadata={
            "source": "admission_notice.txt",
            "department": "Admissions"
        }
    ),
```

```
    Document(
        page_content="The library is open from 8 AM to 6 PM.",
        metadata={
            "source": "library_notice.txt",
            "department": "Library"
        }
    ),

    Document(
        page_content="All students must carry their ID cards.",
        metadata={
            "source": "student_notice.txt"
        }
    )

]


# ========================================
```

```
# DISPLAY TOTAL DOCUMENTS
# ========================================

print("STUDENT INFORMATION PACKET")
print("==========================")

print("Total documents:", len(documents))



# ========================================
# DISPLAY EVERY DOCUMENT
# ========================================

for number, document in enumerate(
    documents,
    start=1
):

    print("\nDOCUMENT", number)
```

```
print(
    "Content:",
    document.page_content
)

print(
    "Source:",
    document.metadata.get(
        "source",
        "Unknown"
    )
)

print(
    "Department:",
    document.metadata.get(
        "department",
        "Not specified"
    )
)
```

8. Important Learning Conclusions

1. **LLMs do not automatically receive the contents of our private external files.**
2. **LangChain provides the Document class to represent text and related metadata.**
3. **page_content contains the actual text.**
4. **metadata maintains information describing that text.**
5. **One physical file can be represented by one or multiple Document objects.**
6. **Multiple Document objects can be maintained together in a Python list.**
7. **Creating Document objects manually is different from reading external files using Document Loaders.**

9. Multiple-Choice Questions

9.1 MCQ 1 — page_content

Question: What does page_content store?

A. Only the filename

B. Actual text content

C. API key

D. Model parameters

Correct Answer: B. Actual text content

Explanation: page_content stores the actual text represented by a Document.

9.2 MCQ 2 — metadata

Question: What is the main purpose of metadata?

A. Train the LLM

B. Generate AI responses

C. Maintain descriptive information

D. Execute Python programs

Correct Answer: C. Maintain descriptive information

Explanation: Metadata stores information describing the content, such as source, department and author.

9.3 MCQ 3 — Document Creation

Question: Does manually creating a Document automatically read a text file?

A. Yes

B. No

C. Only for PDF files

D. Only when an API key is available

Correct Answer: B. No

Explanation: Manually creating a Document does not open or read a file.